

Contents

Card Shuffle Simulator – User Guide	1
1. Introduction	1
2. Installation & Requirements	1
3. Interactive Prompts (in order)	2
4. Shuffle Algorithms	2
5. Card Encoding & Display	3
6. Logging System	3
7. Resuming a Run	3
8. Progress Reporting	4
9. Stopping Conditions	4
10. Performance Tips	4
11. Troubleshooting	4
12. File Naming Convention	5
13. Version History (summary)	5
14. License & Disclaimer	5

Card Shuffle Simulator – User Guide

Version 1.2.0

Date: 2026-08-22

1. Introduction

The Card Shuffle Simulator is a command-line tool that models realistic card shuffling. It is useful for:

- Studying the mixing properties of the Gilbert-Shannon-Reeds (GSR) model
- Experimenting with multi-deck and reduced-suit configurations
- Measuring how many shuffles are required before a deck returns to its original order
- Long-running Monte-Carlo style simulations with full audit trails

The program is written in pure Python 3 and depends only on the standard library.

2. Installation & Requirements

1. Ensure Python 3.8+ is installed:

```
python3 --version
```

2. Copy `card_shuffle_simulator.py` to any directory.

3. Make it executable (optional):

```
chmod +x card_shuffle_simulator.py
```

4. Run:

```
python3 card_shuffle_simulator.py
```

No additional packages are required. On Windows the `resource` module is unavailable; peak-memory reporting is simply omitted.

3. Interactive Prompts (in order)

When you start a new run the program asks the following questions. Press Enter to accept the default shown in brackets.

#	Prompt	Default	Notes
1	Load raw shuffle data from log file and resume shuffling? (y/n)	n	Answer y to resume
2	Number of decks (1–10 recommended, up to 99 accepted)	1	Integer 1–99
3	Include which suits? (S=spades, H=hearts, C=clubs, D=diamonds)	SHCD	Any combination; empty → SHCD
4	Include jokers? (y/n)	n	Adds J1 and J2
5	Initial order: ‘o’ = ordered (new deck), ‘r’ = random	o	
6	Shuffle method (g = GSR, s = simple riffle)	g	
7	Loop until the deck returns to its exact original order? (y/n)	n	If y, the next prompt is skipped
8	How many shuffles to perform?	1000000	Only asked when not looping
9	Show progress every how many shuffles?	0	0 = no intermediate progress
10	Log raw shuffle data to file? (y/n)	n	Creates the verbose log

When resuming (y to the first question) you are asked only for the log-file name; all other parameters are restored from the log.

4. Shuffle Algorithms

4.1 Gilbert-Shannon-Reeds (GSR) – default

1. Cut the deck at a position drawn from a binomial distribution $\text{Binomial}(n, \frac{1}{2})$.
2. Interleave the two halves by repeatedly choosing the next card from the left or right packet with probability proportional to the remaining cards in that packet.

This is the classic mathematical model of a “good” riffle shuffle.

4.2 Simple riffle

1. Cut the deck exactly in half (or as close as possible).
2. Alternate cards from the two halves, randomly deciding which half starts.

Faster but less realistic than GSR.

5. Card Encoding & Display

- Internally every card is a single integer.
- Display format:
 - Single deck: AS, 2H, 1C (ten of clubs), KD, J1, J2
 - Multi-deck: a zero-padded numeric prefix is added (2AS, 10KC, ...)
- The ten of any suit is always shown as the single character 1.
- Jokers are always J1 and J2 (never “JokerA/B”).

When fewer than four suits are selected the joker boundary is calculated dynamically so that jokers never collide with ordinary cards.

6. Logging System

6.1 Abridged log (always created)

Filename pattern:

`card_shuffle_simulator_YYYYMMDDHHMMSS_abridged.log`

Contents (overwritten on every progress update and at the end):

- Version
- Last visible shuffle number
- UTC timestamp
- All original settings
- The exact initial deck (raw integers)
- The most recent deck state that was shown on screen, plus cumulative elapsed time

Because only the latest state is kept, the file stays small even for multi-million-shuffle runs.

6.2 Verbose log (optional)

Filename pattern:

`card_shuffle_simulator_YYYYMMDDHHMMSS.log`

Contents:

- Header with all parameters
- **Initial:** ...
- One line per shuffle:
 Shuffle N: 12 0 45 ... **elapsed:** 123.4567

When you resume from a verbose log the program opens it in **append** mode so the audit trail continues uninterrupted.

7. Resuming a Run

1. Answer y to the first prompt.
2. Enter the name of either an abridged or a full verbose log.
3. The program restores:
 - All original settings
 - The last recorded deck order
 - The cumulative elapsed time

4. Shuffling continues from that point.
5. If the target number of shuffles (or the “return to original”) has already been reached, the program reports completion and exits.

You may manually edit the “How many shuffles” value inside an abridged log; the next resume will honour the new target.

8. Progress Reporting

When `Show progress every how many shuffles?` is greater than zero the program prints:

```
Shuffle 100000 | Elapsed: 58.9s | Peak memory: 10.5 MB
AS 2H ... J2
```

- The final shuffle of a fixed-count run does **not** produce an intermediate progress line (the final summary is printed instead).
 - Peak memory is sampled only every 10 progress intervals to keep overhead low.
-

9. Stopping Conditions

The simulation ends when any of the following occurs:

- The requested number of shuffles has been reached (fixed-count mode).
 - The deck returns to its exact original order (always checked, even if you answered “n” to the loop question). An explicit alert is printed.
 - The user interrupts with Ctrl-C (the verbose log handle is closed cleanly via a `finally` block).
-

10. Performance Tips

- Prefer the GSR method for scientific accuracy; the simple riffle is faster for pure volume testing.
- Set progress interval to 0 (or a large number) when running tens of millions of shuffles to minimise I/O.
- The verbose log can become large; disable it unless you need a complete audit trail.
- Multi-deck + jokers increases both memory and CPU cost linearly.

Typical throughput on modern hardware (single deck, GSR, no progress output):

- ~ 15 000–25 000 shuffles per second (varies with Python version and CPU).
-

11. Troubleshooting

Symptom	Likely cause	Remedy
<code>NameError:</code> <code>cards_per_suit_set</code>	Older script version	Use v1.2.0 or later
Jokers display as ordinary cards	Reduced-suit run with pre-1.1 code	Upgrade
Resume does not append to verbose log	Pre-1.2.0 code	Upgrade – handle is now opened in append mode
<code>resource</code> module missing on Windows	Expected	Peak-memory line shows “N/A”

Symptom	Likely cause	Remedy
Very slow progress	Progress interval too small + verbose logging	Increase interval or disable verbose log

12. File Naming Convention

All log files use UTC timestamps of the form YYYYMMDDHHMMSS:

```
card_shuffle_simulator_20260822193407.log
card_shuffle_simulator_20260822193407_abridged.log
```

13. Version History (summary)

- **1.2.0** (2026-08-22) – Resume from full verbose log correctly appends; version metadata; documentation; guaranteed `cards_per_suit_set` in all execution paths.
 - **1.1.x** – Dynamic suit counts, robust input validation, deque-based GSR, abridged logging, cumulative elapsed time, original-order alert.
 - **1.0** – First interactive implementation.
-

14. License & Disclaimer

This software is provided as-is with no warranty of any kind. Use it freely for research, education, or entertainment.

End of User Guide